CURSO COMPLETO · TRADUCIDO Y EXPLICADO

Cómo hacer ingeniería inversa de cualquier programa con Claude Opus 5.5

Versión en español, con glosario, explicaciones sencillas de cada paso y todos los prompts listos para copiar.

Artículo original	“How to Reverse Engineer Anything with Opus 5.5 (Full-Course)”
Autor	Movez (@0xMovez) en X
Fecha	12 de octubre de 2026
Enlace	x.com/0xMovez/status/2109604520770548146

Cómo leer esta guía

Cada paso tiene tres partes: lo que **explica el autor** (contado en español, sin tecnicismos innecesarios), los **comandos y prompts** (los prompts están traducidos al español; puedes pegarlos así en Claude) y un recuadro **“En pocas palabras”** que resume la idea clave.

Si una palabra te suena a chino, búscala en el **glosario** de la página siguiente. Al final hay una sección **“Notas antes de lanzarte”** con avisos que no están en el artículo (cifras sin verificar y legalidad en España).

Glosario rápido

Los términos que aparecen en el curso, explicados en una línea.

Término	Qué significa
Ingeniería inversa	Analizar un programa ya terminado para entender cómo funciona por dentro, sin tener su código original.
Binario / ejecutable	El archivo final que ejecuta el ordenador: instrucciones de máquina, ilegibles para una persona.
Código fuente	El código original que escribió el programador, en un lenguaje legible (C, Swift, JavaScript…).
Ensamblador (assembly)	Las instrucciones de máquina escritas como texto. Legible, pero de muy bajo nivel (mover datos, saltar, sumar).
Desensamblador	Herramienta que convierte el binario en ensamblador.
Descompilador	Va un paso más allá: reconstruye código parecido a C (pseudocódigo) a partir del ensamblador.
Pseudocódigo	Código aproximado y legible, pero con nombres genéricos sin sentido como `var_28` o `sub_100012ab0`.
Hopper / Ghidra / IDA	Los tres “motores” que desensamblan y descompilan. Ghidra es gratuito (lo publicó la NSA); Hopper es de pago y va muy bien en Mac; IDA es el profesional y caro.
MCP	Model Context Protocol: un estándar para conectar herramientas externas a un agente de IA, que puede “llamarlas” como funciones.
REA	“Reverse Engineer Anything”: la herramienta que hace de puente entre Claude y los motores anteriores, vía MCP y línea de comandos.
Strings (cadenas)	Textos legibles guardados dentro del binario: URLs, mensajes de error, nombres de funciones.
Xrefs	Referencias cruzadas: cuántos sitios del programa llaman a una función. Muchas xrefs = función importante.
Electron	Forma de hacer apps de escritorio con JavaScript y HTML (Notion, Slack, VS Code). Su código JS suele estar casi a la vista.
Mach-O / ELF / PE	Formatos de programa nativo de macOS / Linux / Windows.
.NET, Go, APK, WASM	.NET: plataforma de Microsoft (fácil de descompilar). Go: lenguaje de Google. APK: app de Android. WASM: código compilado que corre en el navegador.
IPC	Comunicación entre procesos. En Electron, la conversación entre la ventana y el proceso principal de la app.
Ofuscación	Técnicas para enredar el código a propósito y que sea difícil de analizar.
Struct	Estructura de datos: un grupo de campos que van juntos (p. ej. “Config” con endpoints, modo de autenticación, reintentos).
Sala limpia (clean room)	Método para reconstruir un programa legalmente: alguien describe QUÉ hace; otra persona, que nunca vio el original, lo programa solo a partir de esa descripción.
Paridad	Que tu reconstrucción se comporte igual que el original ante las mismas entradas.

Término	Qué significa
Arnés de pruebas	Conjunto automático de tests que mete entradas en dos programas y compara lo que devuelven.
Esfuerzo (effort)	Cuánto “piensa” Opus 5.5 antes de responder: low, medium, high, xhigh, max. Más esfuerzo = más calidad, más tiempo y más coste.
Tokens	La unidad en la que se mide (y se cobra) el texto que procesa el modelo.

El punto de partida

El autor arranca con una escena típica: alguien abre un binario en un editor hexadecimal, hace scroll veinte minutos y lo deja. No sabe qué herramienta abre el archivo, qué pedirle a la IA para sacar la lógica, ni cómo pasar de un muro de ensamblador a algo que pueda reconstruir. Su promesa: en 11 pasos, convertir a Claude Opus 5.5 en una máquina capaz de “abrir” cualquier programa.

Su tesis central: **la ingeniería inversa no va de crackear programas, sino de entender algo lo bastante bien como para construir tu propia versión.**

Qué desencadenó el artículo

- **7 de octubre de 2026:** Amjad Masad, CEO de Replit, publicó que lo que está pasando con la ingeniería inversa y la descompilación con IA es una locura, y que pronto todo el software será, en la práctica, de código abierto.
- Un desarrollador, **@morluto**, respondió enseñando **REA**: un conjunto de herramientas para que los agentes de IA manejen Hopper y Ghidra. Según el autor, superó las 110.000 estrellas en GitHub en pocos días. Repositorio: `github.com/morluto/rea`.
- Esa misma semana, según el autor, el proyecto **WordCraft** publicó una reconstrucción en Rust de herramientas de Microsoft y Adobe, hecha observando cómo se comportan y leyendo especificaciones públicas de formatos de archivo, sin tocar nunca su código. Web: `getartcraft.com/apps`.

Entre los pasos, el artículo intercala tweets de otros usuarios: Vaishnavi (@_vmlops) presentando REA; Md Ismail Šojal (@0x0SojalSec) hablando de IDA Pro con IA y de `ghidra-mcp` (más de 200 herramientas para que agentes usen Ghidra, en `github.com/bethington/ghidra-mcp`); Victor (@victor_explore) con una lista de 10 repos útiles (encabezada por Ghidra, unas 84.000 estrellas); y Yum (@yuhasbeentaken), que tras 12 horas probando REA cuenta que puede darle una orden general tipo “reconstruye esta app”, o pasarle un driver de impresora o un binario antiguo de Windows, y dejar que el agente decida qué herramientas usar.

El mapa del curso

Paso	Qué haces	Herramienta clave
01 · El motor	Entender cómo Claude puede “leer” un binario	REA + Hopper / Ghidra / IDA
02 · Instalación	Instalar REA y un desensamblador	`npx rea-agents setup`
03 · El enrutador	Identificar qué tipo de programa tienes delante	Prompt de entrada
04 · Reconocimiento	Mapa general, textos y quién llama a quién	`binary_overview`, `search`, `xrefs`
05 · Descompilar	Convertir una función en pseudocódigo	`rea decompile`
06 · Anotar	Que Opus ponga nombres y explique la función	Prompt de anotación
07 · Ofuscación	Desenredar código protegido a propósito	Prompt a esfuerzo max
08 · Enjambre	Repartir cientos de funciones entre varios chats	Controlador + workers

Paso	Qué haces	Herramienta clave
09 · Sala limpia	Escribir la especificación sin copiar código	Prompt de especificación
10 · Crafting Apps	El ejemplo real: 12 apps reconstruidas	WordCraft por MCP
11 · Verificar y publicar	Demostrar que funciona igual y compartirlo	Arnés de 50 pruebas

PASO 01

El motor

Cómo un agente de IA lee un binario

Opus 5.5, por sí solo, no puede ejecutar un desensamblador: solo lee texto y escribe texto. La ingeniería inversa con un agente funciona porque hay una herramienta en medio que carga el archivo en un motor de desensamblado de verdad, convierte los resultados en funciones que el modelo puede llamar y le deja hacer preguntas. Esa herramienta es **REA**, y el motor que hay debajo es **Hopper, Ghidra o IDA**.

Claude Opus 5.5	↔	REA (vía MCP)	↔	Hopper / Ghidra / IDA	↔	Binario
Piensa y pregunta		Traduce preguntas en análisis		Desensambla y descompila		El programa a estudiar

El bucle que se repite:

- 1. El modelo pide a REA que inspeccione algo del programa.
- 2. REA devuelve lo que encuentra **con la evidencia exacta**: direcciones de memoria, bytes, pseudocódigo y desde dónde se llama.
- 3. El modelo usa esos hallazgos para hacer la siguiente pregunta, o para escribir una reimplementación y probarla.

Todo se ejecuta en tu ordenador. Al modelo solo se envía lo que necesita para razonar, y cada conclusión se etiqueta como **observación** (lo he visto), **inferencia** (lo he deducido) o **desconocido**.

> **En pocas palabras**
> Claude es el cerebro, REA son las manos y Hopper/Ghidra/IDA son los ojos que ven dentro del binario. Sin REA, Claude no puede “abrir” un programa; con REA, puede interrogarlo pregunta a pregunta.

PASO 02

La instalación

rea-agents + un desensamblador

La configuración registra el servidor MCP de REA en tu agente e instala las instrucciones de trabajo que lo acompañan. Es un solo comando, te enseña todos los cambios antes de escribir nada y hace copia de seguridad de lo que toca. El desensamblador lo pones tú: **Hopper** en macOS es lo más fácil, **Ghidra** es gratis y funciona en cualquier sistema, e **IDA** si ya lo tienes.

TERMINAL

```
# 1. Registrar REA en tu agente (Claude Code, Codex, Cursor, Gemini CLI...)
npx rea-agents setup        # muestra los cambios previstos y pide permiso

# 2. Un motor de análisis profundo. Hopper es el más cómodo en macOS
#    (setup puede instalarlo si lo apruebas). Ghidra e IDA los traes tú.

# 3. Instalar la CLI "rea" para usarla en la terminal y en scripts
npm install --global rea-agents
rea --help

# 4. Reinicia el agente y comprueba que las herramientas están activas
claude
> /mcp
  rea: connected
    open_binary, binary_overview, analyze_function, decompile,
    list_procedures, search, xrefs, trace, inspect_artifact,
    analyze_javascript_application, inspect_managed_artifact ...
```

Requisitos: Node.js 22 o superior y npm. El análisis nativo necesita uno de los tres motores; para inspeccionar JavaScript y .NET no hace falta ninguno. Mantén REA actualizado, porque sacan correcciones a menudo.

Las “reglas de la casa” para Claude

El autor propone meter estas reglas en el archivo `CLAUDE.md` del proyecto, para que el agente las cumpla siempre:

CLAUDE.md

```
# Reglas de ingeniería inversa (CLAUDE.md)

## Autorización
- Analiza solo objetivos que sean míos o que esté autorizado a examinar.
  Indica la base de esa autorización una vez.
- Etiqueta cada afirmación: observación (lo vi), inferencia (lo deduje)
  o desconocido.
- Cita la evidencia: dirección, bytes, pseudocódigo o punto de llamada.
  Nada de afirmaciones sin fuente.

## Flujo de trabajo
- Empieza por el resumen más pequeño, no por un volcado completo. Amplía solo
  cuando haya un hueco concreto.
- Reutiliza la sesión abierta; nunca repitas un análisis idéntico.
- Separa los hechos estáticos de los hechos en ejecución. Nunca des por
  ejecutado algo que solo has leído.

## Reconstrucción
- Construye a partir del COMPORTAMIENTO y de especificaciones públicas, nunca
  a partir de código o recursos copiados.
- Lleva un registro de obligaciones: cada comportamiento recuperado necesita
  un test que pase antes de darlo por "hecho".
```

El nivel de esfuerzo de Opus 5.5

Opus 5.5 tiene cinco niveles de esfuerzo. Según el autor, **es el mando que más influye en la calidad de todo el curso**:

Nivel	Cuándo usarlo
low	El artículo no le da ningún uso.
medium	Inspeccionar el programa y moverte por él.
high	Anotar funciones normales.

Nivel	Cuándo usarlo
xhigh	Es el que pide el prompt de anotación del paso 06.
max	Código ofuscado y cualquier cosa con mucha aritmética de punteros (cálculos con direcciones de memoria, típicos de C).

En pocas palabras

Un comando instala REA, tú añades un desensamblador (Ghidra si quieres gratis) y pones unas reglas en `CLAUDE.md` para que Claude cite pruebas y no copie código. Sube el esfuerzo solo cuando el código lo merezca.

PASO 03

El enrutador

Cada tipo de programa, su herramienta

El primer error del principiante es sacar el desensamblador cuando el programa es una app de Electron cuyo JavaScript está ahí, a la vista. REA dirige cada objetivo a la herramienta adecuada (el autor habla de 20 tipos de objetivo), y tu primer prompt debe hacer lo mismo: di qué archivo es, qué función te interesa y deja que el agente elija por dónde entrar.

PROMPT · PRIMER CONTACTO

```
Quiero entender cómo funciona [FUNCIÓN] en [APP], que está en [RUTA ABSOLUTA].

Primero, dime qué tipo de objetivo es (nativo, Electron/JS, .NET, Go, APK, WASM)
y elige la herramienta de REA adecuada para abrirlo. Todavía no empieces el
análisis profundo.

Después dame un mapa en un párrafo: con qué está construida la app, qué tamaño
tiene y dónde mirarías primero para encontrar [FUNCIÓN]. Separa observaciones,
inferencias y desconocidos, y cita la evidencia de todo lo que afirmes.
```

Si el objetivo es…	Qué pasa
Una app Electron (JavaScript)	REA devuelve módulos, imports y límites de IPC en segundos, sin desensamblador.
Un nativo Mach-O (Mac) o ELF (Linux)	REA abre una base de datos de Hopper: el camino más difícil, pero el que llega más hondo.

En pocas palabras

Antes de nada, averigua qué tipo de programa es. Saber si estás en el camino “fácil” (JavaScript) o en el “nativo” te ahorra horas.

PASO 04

Reconocimiento

Resumen general, textos y el grafo de llamadas

Todavía no abras código. Primero pide a REA un mapa del archivo entero: en qué tipo de procesador corre, cuántas funciones tiene y todos los textos legibles que lleva dentro. Esos textos son tu pista más rápida: URLs, mensajes de error y nombres de funciones te dicen qué hace la app antes de leer una sola línea de código.

EJEMPLO DE SESIÓN

```
rea.open_binary      { path: "/Applications/Target.app/Contents/MacOS/Target" }
rea.binary_overview  {}
  arch:        arm64
  functions:   1,284 identified
  segments:    __TEXT __DATA __LINKEDIT
  entry:       _main @ 0x1000051c0

rea.search           { query: "http" }   # textos + dónde se usan en el código
  0x10004a2c0  "https://api.target.app/v3/sync"
  0x10004a300  "Authorization: Bearer "
  0x10004a330  "offline cache miss, falling back"

rea.xrefs            { address: "0x100012ab0" }   # ¿quién llama a esto?
  147 references  ->  core config loader, called everywhere
```

Cómo leer esa salida

Línea	Qué te está diciendo
`arch: arm64`	El programa es para procesadores ARM de 64 bits (los Mac con chip Apple).
`functions: 1,284`	El motor ha identificado 1.284 funciones.
`segments`	`__TEXT` es el código, `__DATA` los datos, `__LINKEDIT` información para enlazar librerías.
`entry: _main`	El punto por el que arranca el programa.
Resultados de `http`	La app sincroniza con una API propia, se autentica con un token “Bearer” y tiene una caché para trabajar sin conexión. Todo eso, sin leer código.
`147 references`	La función de esa dirección se llama desde 147 sitios: es el cargador central de configuración.

REA también cuenta cuántas veces se llama a cada función (sus **xrefs**). Una función llamada 147 veces es el corazón de la app y merece tu mejor esfuerzo; una llamada dos veces es un ayudante menor que puedes ignorar hasta que importe. **Empieza por arriba de esa lista y ve bajando.**

> **En pocas palabras**
> Primero el mapa, luego el detalle. Los textos te dicen QUÉ hace la app; las xrefs te dicen DÓNDE está lo importante.

PASO 05

Descompilar

Una función convertida en pseudocódigo

Ahora elige una de esas funciones muy llamadas y conviértela en algo legible. El descompilador toma las instrucciones de máquina y las reescribe como código parecido a C. Queda feo (nombres como `var_28`, lógica enredada, conversiones de tipo por todas partes), pero se puede leer, y eso es todo lo que Opus 5.5 necesita. Puedes hacerlo desde la terminal para guardar el resultado, o desde el chat.

TERMINAL

```
# Descompilar una función por su dirección y guardarla como JSON
rea decompile /Applications/Target.app/.../Target 0x100012ab0 \
  --provider hopper --json > fn_config_loader.json

# O buscar una función por su nombre y seguir sus referencias
rea function /path/to/Target parseConfig  --provider hopper --json
rea xrefs    /path/to/Target 0x100012ab0  --provider hopper --json
rea trace    /path/to/Target "sync"       --provider hopper --json
```

PSEUDOCÓDIGO QUE DEVUELVE EL DESCOMPILADOR

```
// 0x100012ab0  xrefs: 147
void sub_100012ab0(void* arg0) {
    void* var_20 = objc_msgSend(cls_NSJSONSerialization,
        sel_JSONObjectWithData_options_error, arg0, 0x0, 0x0);
    if (var_20 != 0x0) {
        void* var_28 = objc_msgSend(var_20, sel_objectForKey, @"endpoints");
        void* var_30 = objc_msgSend(var_20, sel_objectForKey, @"auth_mode");
        void* var_38 = objc_msgSend(var_20, sel_objectForKey, @"retry_count");
        objc_msgSend(g_sharedConfig, sel_setEndpoints, var_28);
        objc_msgSend(g_sharedConfig, sel_setAuthMode, var_30);
        objc_msgSend(g_sharedConfig, sel_setRetryCount,
            objc_msgSend(var_38, sel_intValue));
    }
}
```

Traducido a lenguaje humano: la función recibe unos datos, los convierte de JSON a un objeto; si eso sale bien, lee tres claves (`endpoints`, `auth_mode`, `retry_count`) y las guarda en la configuración global de la app (`g_sharedConfig`). Si el JSON está mal, no hace nada y no avisa.

El ejemplo de REA con el juego DX-Ball: siguieron una llamada de sonido del juego hasta la función que decide el balance izquierda-derecha del audio y convirtieron el pseudocódigo a C. Según el artículo, el resultado pasa los 3.205 casos de prueba originales y reproduce los 63 bytes compilados uno a uno. Lo tienes completo en `rea.tools/showcase`.

> **En pocas palabras**
> El descompilador te da código feo pero legible. No hace falta que lo entiendas tú: hace falta que lo entienda Opus (paso 06).

PASO 06

Anotar

Que Opus ponga nombre a cada variable

Ese código feo es solo la mitad del trabajo. Ahora pásaselo a Opus 5.5 y pídele que lo explique y lo limpie: renombra las variables crípticas, resume en una frase para qué sirve la función, identifica el patrón que sigue e incluso señala fallos que dejó la app original.

PROMPT · ANOTACIÓN

```
Aquí tienes el pseudocódigo descompilado de una función con 147 referencias
cruzadas.

Para esta función:
1. Renombra cada variable con un nombre con significado según cómo se usa.
2. Explica su propósito en una frase.
3. Enumera las estructuras de datos que lee y escribe.
4. Nombra el patrón (singleton, factory, máquina de estados, observer...).
5. Reescríbela como Swift limpio e idiomático (o el lenguaje que sugieran las
   convenciones de llamada), con comentarios de documentación.
6. Señala cualquier falta de manejo de errores, validación de entradas o
   comprobación de seguridad.

Piénsalo con esfuerzo xhigh antes de responder: razona qué es cada valor antes
de decidir los nombres. Marca todo aquello de lo que no estés seguro.

[pega aquí el pseudocódigo]
```

Los patrones que menciona el prompt

Patrón	Qué es
Singleton	Un objeto del que solo existe una copia en toda la app (como `g_sharedConfig`).
Factory	Una función que fabrica objetos de distintos tipos según lo que le pidas.
Máquina de estados	Algo que pasa de un estado a otro según eventos (desconectado → conectando → conectado).
Observer	Objetos que se “suscriben” para enterarse cuando algo cambia.

Para que veas el resultado que buscas, este sería un ejemplo de salida para el pseudocódigo del paso 05 (**ilustración propia, no está en el artículo**):

EJEMPLO ILUSTRATIVO · SWIFT LIMPIO

```
/// Carga la configuración global de la app a partir de datos JSON.
/// Patrón: singleton (AppConfig.shared es la única instancia).
func loadConfig(from data: Data) {
    guard let json = try? JSONSerialization.jsonObject(with: data)
            as? [String: Any] else {
        return  // AVISO: falla en silencio si el JSON es inválido
    }
    let config = AppConfig.shared
    config.endpoints  = json["endpoints"] as? [String] ?? []
    config.authMode   = json["auth_mode"] as? String
    config.retryCount = (json["retry_count"] as? Int) ?? 0  // sin validar
}
```

No todos los objetivos necesitan desensamblador. En el ejemplo de REA con **Notion**, localizan la función del portapapeles en el JavaScript de la app, la siguen a través de las capas de Electron hasta el proceso principal y muestran cómo construye el portapapeles enriquecido: un rastreo completo sin ver ni una línea de ensamblador.

> **En pocas palabras**
> El descompilador te dice QUÉ hace el código. Opus te dice POR QUÉ, y te da los nombres para reconstruirlo.

PASO 07

Ofuscación

El benchmark de Tigress

Algunas apps están hechas para resistirse justo a esto. Su código está revuelto a propósito: la lógica desordenada, los textos cifrados y ramas falsas añadidas para hacerte perder el tiempo. Eso es la **ofuscación**, y es el único terreno donde incluso el mejor modelo sufre.

Elastic Security Labs lo midió: enfrentó a Claude Opus con **Tigress** (un ofuscador) en 20 tareas. El artículo original mostraba las cifras en una imagen que no venía en el texto; el único dato que menciona por escrito es que **en ofuscación de varias capas se resolvió el 0%**. De esos números saca dos lecciones prácticas:

- **Un intento fallido cuesta más o menos el doble que uno exitoso**, porque el modelo se esfuerza cada vez más antes de rendirse. Pon un límite de gasto por función y corta pronto.
- **Limpia la basura del pseudocódigo antes de enviarlo.** El autor menciona una herramienta gratuita, Caveman, que quita alrededor del 75% del relleno, para que el modelo gaste su atención en la lógica real.

Las cinco técnicas que te vas a encontrar

Técnica	Qué hace (y cómo se ve)
Aplanamiento del flujo de control	Convierte la lógica normal en un gran bucle con un “despachador” (un switch) que salta entre bloques. Se pierde el orden natural del código.
Predicados opacos	Condiciones que siempre dan el mismo resultado pero no lo parecen, p. ej. `(x*(x+1)) % 2 == 0` (siempre es verdad). Sirven para crear ramas falsas.
Código muerto	Bloques que nunca se ejecutan, metidos solo para despistar.
Cifrado de cadenas	Los textos se guardan cifrados y se descifran al ejecutar. Así la búsqueda de textos del paso 04 no encuentra nada.
División de funciones	Una función lógica partida en muchas funciones pequeñas para que no se vea el conjunto.

Estas explicaciones de cada técnica son mías; el artículo solo las nombra dentro del prompt.

PROMPT · DESOFUSCAR

```
Esta función parece ofuscada. Trabaja con esfuerzo max.

1. Identifica las técnicas presentes (aplanamiento del flujo de control,
   predicados opacos, código muerto, cifrado de cadenas, división de funciones)
   y señala dónde aparece cada una en este código concreto.
2. Resuélvelas: elimina las ramas muertas, evalúa los predicados opacos,
   deshaz el aplanamiento del despachador y descifra las cadenas si la clave
   se puede recuperar.
3. Muestra la función limpia que queda.
4. Puntúa tu confianza de 1 a 10. Por debajo de 6, dime exactamente qué partes
   son inciertas y qué contexto de funciones vecinas lo resolvería. No adivines
   en silencio.

[pega aquí el pseudocódigo ofuscado]
```

En pocas palabras

Contra la ofuscación, esfuerzo max, presupuesto limitado y código limpio de relleno. Si es ofuscación de varias capas, asume que probablemente no saldrá y no te quemes ahí.

PASO 08

El enjambre

Un controlador que dirige a varios workers

Un solo chat no puede leer bien 500 funciones: olvida lo que vio, llama a lo mismo de tres formas distintas y empieza a contradecirse. La solución es un pequeño equipo: **un chat controlador** que reparte el trabajo y **varios chats worker** que se encargan de una función cada uno. El controlador mantiene la visión global y junta las piezas. No hay que programar nada especial: estos dos prompts son toda la configuración.

PROMPT · CONTROLADOR

```
Eres un controlador de ingeniería inversa. El mapa de funciones está en
functions.json (address, name, size, xref_count).

1. Agrupa las funciones por cercanía en el grafo de llamadas; ordena los grupos
   por número de xrefs.
2. Envía cada grupo a un worker. Nunca analices una función tú mismo.
3. Incluye en las instrucciones de cada worker los structs ya resueltos, para
   que los nombres sean coherentes.
4. Cuando un worker devuelva una confianza menor que 6, programa una segunda
   pasada con contexto de sus funciones vecinas ya resueltas.
5. Lleva la cuenta del gasto acumulado de tokens e infórmalo tras cada grupo.
```

PROMPT · WORKER

```
Eres un worker. Recibes una función descompilada y los structs resueltos hasta
ahora. Devuelve exactamente: address, purpose, confidence (1-10), structs_used,
structs_new, clean_source, flags. Reutiliza los nombres de structs existentes;
nunca inventes un segundo nombre para un struct que el controlador ya te dio.
Solo una función. Para cuando termines.
```

Los nombres de campo (`address`, `purpose`, `confidence`…) se dejan en inglés a propósito: son las claves del formato de respuesta y conviene que sean siempre las mismas.

En pocas palabras

Como un jefe de obra: el controlador no pone ladrillos, reparte tareas y se asegura de que todos llamen igual a las mismas cosas.

PASO 09

Sala limpia

Las cuatro reglas que lo mantienen legal

Reconstruir en **sala limpia** significa copiar lo que la app **hace**, nunca el código que lleva dentro. Si lo haces bien, lo que publicas es completamente tuyo; si lo haces mal, cargas con los derechos de autor de otro. Cuatro reglas te mantienen en el lado seguro (según el autor, el equipo de WordCraft las comprueba automáticamente en cada compilación):

Regla	Qué implica
1. Observa, nunca copies	Usa el programa como caja negra, mira lo que hace y haz capturas solo de su propia ventana. Nunca leas, desensambles ni pegues su código. No copies textos más allá de los nombres de las funciones.
2. Especificaciones, no fuentes	Los formatos de archivo salen de estándares públicos (OOXML, el formato de .docx/.xlsx, es ECMA-376 / ISO 29500; más las especificaciones publicadas de RTF y ODF), nunca de leer el lector de archivos de un competidor ni código con licencia GPL/LGPL.
3. Solo recursos originales	Cada icono, fuente, plantilla y sonido es dibujado por código, de dominio público o con licencia abierta, y se apunta en un archivo de atribuciones. Si no, una comprobación automática hace fallar la compilación.
4. Separa los dos trabajos	Un agente recupera el comportamiento y escribe la especificación. Otro contexto distinto implementa SOLO a partir de esa especificación, sin ver nunca el análisis original. Esa separación es la frontera de la sala limpia.

Sobre la regla 2: GPL y LGPL son licencias libres "contagiosas": si copias ese código, puedes quedar obligado a publicar el tuyo bajo la misma licencia.

PROMPT · ESPECIFICACIÓN DE SALA LIMPIA

```
A partir del análisis completado, escribe una especificación de sala limpia
para una reimplementación.

Describe SOLO el COMPORTAMIENTO. Incluye: todas las interfaces públicas, el
modelo de datos, cada funcionalidad visible para el usuario como requisito
comprobable, los formatos de archivo que lee y escribe (según el
comportamiento observado y la especificación pública) y cómo maneja los
errores.

NO incluyas: direcciones u offsets, nombres de variables del descompilado,
ensamblador ni nada copiado del original. Un desarrollador que nunca haya
visto el binario debe poder implementar una versión compatible solo con este
documento.

Entrega markdown con estas secciones: Arquitectura, Modelo de datos, API
pública, Formatos de archivo, Casos de prueba.
```

Trabajar a partir de una descripción no es solo una protección legal: te deja tomar las decisiones correctas para tu lenguaje y tus necesidades, en vez de heredar las que tomó el autor original hace décadas. Según el artículo, WordCraft es más rápido que Word en algunos puntos precisamente porque no copió sus tripas.

> **En pocas palabras**
> Un chat analiza y escribe QUÉ hace el programa. Otro chat, que no ha visto nada del análisis, lo programa desde cero. Así lo que sale es tuyo.

PASO 10

Crafting Apps

Doce herramientas profesionales, un mismo método

Según el autor, WordCraft no es un caso aislado: es una de las **doce apps** del equipo ArtCraft, cada una una reconstrucción en sala limpia de una herramienta por la que ya pagas, escritas desde cero en **Rust** y publicadas gratis y en abierto en GitHub (por ejemplo, Photoshop → PhotoCraft e Illustrator → VectorCraft, según otro tweet del propio autor). Su lema es “Doce apps. Un oficio”, y ese oficio es el método del paso 09.

- **Sin Electron:** cada app es nativa en macOS, Windows y Linux, y todas menos FilmCraft funcionan también en el navegador gracias a WebAssembly.
- **Construidas por capas:** cada capa solo usa la que tiene debajo, desde las medidas básicas abajo hasta la ventana en pantalla arriba (el artículo mostraba las capas de WordCraft en una imagen que no venía en el texto).
- **Cada acción es un comando con nombre:** un clic, un atajo de teclado o un agente de IA disparan exactamente lo mismo. Por eso un agente puede manejarlas directamente.

TERMINAL · CLAUDE CODE

```
# Manejar WordCraft por MCP

# Automatización de documentos sin interfaz (headless)
claude mcp add wordcraft -- wordcraft-cli mcp

# O controlar la app de escritorio mientras está abierta
wordcraft --control 7981 &
claude mcp add wordcraft-app -- wordcraft-cli mcp --connect 127.0.0.1:7981

# Herramientas: list_commands, execute, batch, type_text, select_text,
#   inspect_document, render_page, save_document (+ screenshot/click/key en vivo)
> Abre report.docx, pon todos los títulos en 14 pt negrita y exporta a PDF
  ✓ select.all-headings -> 12 selected
  ✓ format.font-size 14 · format.bold
  ✓ export report.pdf (selectable text, bookmarks kept)
```

En pocas palabras

La idea a copiar para tus propios proyectos: arquitectura por capas y cada acción como un comando con nombre. Así tu app la puede usar igual una persona que un agente.

PASO 11

Verificar y publicar

Demostrar la paridad y exponerlo por MCP

Como nunca tuviste el código del original, no puedes compararlo línea a línea. Comparas el **comportamiento**: das las mismas entradas al original y a tu reconstrucción y compruebas que producen lo mismo.

PROMPT · PRUEBAS DE PARIDAD

```
Construye un arnés de pruebas de comportamiento que compare el binario original
con mi reconstrucción.

Genera 50 entradas: 30 normales, 10 límite (vacía, tamaño máximo, unicode,
caracteres especiales) y 10 de error (mal formadas, campos que faltan,
corruptas). Pasa cada una por ambos. Compara la salida byte a byte cuando se
pueda y, si no, por significado. Informa del porcentaje de coincidencia, una
lista de divergencias y la gravedad de cada una (cosmética / de
comportamiento / crítica). Guárdalo en report.md.
```

Ese informe es además la prueba más convincente que puedes enseñar a cualquiera. Después, publica tu reconstrucción como lo hicieron REA y WordCraft: **como servidor MCP**, para que el agente de la siguiente persona pueda manejarla igual que tú manejaste REA. Dale a cada función un nombre y una descripción claros, envuélvela en una herramienta de línea de comandos sencilla y haz que se instale con una sola línea.

Y **prueba sobre la marcha, no al final**: un error que descubres después de haber construido tres capas encima cuesta mucho más de arreglar.

> **En pocas palabras**
> “Creo que funciona igual” no vale. 50 pruebas y un porcentaje de coincidencia, sí.

Seis errores que te hacen perder un fin de semana

Error	Qué hacer en su lugar
Abrir un desensamblador con una app Electron	Si el JavaScript está a la vista, usa `analyze_javascript_application` y sáltate la vía nativa.
Volcarlo todo antes de leer nada	Pide primero un resumen. Sacar la app entera de golpe entierra la lógica real en ruido y dispara la factura.
Leer las funciones en el orden del archivo	Ordénalas por xrefs. La de 147 es la arquitectura; la de 2 puede esperar.
Atascarte con ofuscación de nivel 3	El benchmark resuelve el 0% de la ofuscación multicapa. Pon un presupuesto, guarda el contexto de las funciones vecinas y vuelve más tarde.
Dejar que el modelo copie código a tu reconstrucción	Rompe la sala limpia. Primero la especificación en un contexto; después, la implementación desde esa especificación en otro.
Publicar sin informe de paridad	“Creo que coincide” no es una prueba. 50 tests de comportamiento y un porcentaje de coincidencia, sí.

Conclusión del autor

Instala REA. Lee el binario. Reconstrúyelo como algo tuyo. Identifica el tipo de programa, mapea su superficie, descompila las funciones más llamadas, deja que Opus 5.5 ponga nombre a todo, rompe la ofuscación que merezca la pena, escala con un enjambre, escribe una especificación limpia y publica la reconstrucción por MCP. Once pasos, un modelo y cero código copiado.

Notas antes de lanzarte

Esta sección no forma parte del artículo: son avisos que añado para que lo leas con criterio.

1. Hay cifras y afirmaciones sin verificar

No he comprobado los datos del artículo. Algunos tienen tono claramente promocional: las 110.000 estrellas de REA en días, que WordCraft "reconstruyó Microsoft y Adobe", o el tweet del propio autor diciendo que un programador solitario "destruyó el imperio de Adobe". El autor además promociona su Substack. Antes de invertir tardes en esto, entra en los repositorios y mira actividad real, issues abiertos, licencia y si de verdad hacen lo que se dice.

2. La legalidad en España y la UE no es la de EE. UU.

La primera regla del `CLAUDE.md` (analizar solo lo que es tuyo o estás autorizado a examinar) es la clave. En la UE, la Directiva 2009/24/CE, recogida en España en el artículo 100 de la Ley de Propiedad Intelectual, permite a quien usa legítimamente un programa **observar, estudiar y probar su funcionamiento** para entender las ideas que hay detrás; eso no lo puede prohibir la licencia. **Descompilar** (reproducir y traducir su código) solo está permitido en supuestos estrictos, básicamente para conseguir que tu programa sea interoperable con él, y la información obtenida no puede usarse para crear un programa sustancialmente similar.

Fíjate en una tensión del propio artículo: los pasos 04 a 08 desensamblan y descompilan, pero la regla 1 de la sala limpia dice "nunca desensambles". WordCraft, según el autor, se hizo solo observando el comportamiento. Esa vía (caja negra + especificaciones públicas) es la que está más claramente cubierta por la ley europea; la de descompilar para luego reconstruir un producto competidor es mucho más arriesgada. Esto no es asesoramiento legal: si vas a sacar un producto comercial basado en esto, consúltalo con un abogado de propiedad intelectual.

3. El coste se dispara rápido

Esfuerzo max, enjambres de workers y binarios con miles de funciones consumen muchos tokens. Aplica los consejos del propio autor: resumen antes que volcado, prioriza por xrefs, presupuesto por función y que el controlador informe del gasto tras cada grupo.

4. Partes del artículo que no venían en el texto

El original incluía imágenes (la gráfica del benchmark de Tigress, el diagrama de capas de WordCraft y una tabla de resultados al inicio) que no aparecían en el texto copiado, así que no están en esta guía.

> **Por dónde empezar (sin riesgo)**
> Instala REA con Ghidra (gratis) y pruébalo con un programa tuyo: compila una pequeña app propia en C o Go, o empaqueta una app Electron que hayas hecho tú, y haz los pasos 03 a 06 sobre ella. Así aprendes el flujo completo sin tocar software de terceros.